HTML5 Tutorial: Learn by Building

A hands-on guide to HTML structure, text, media, and forms. Every tag is explained in a sentence and then shown in working code.

How to use this guide

1. Create a folder called `html-practice` and a file inside it called `index.html`.
2. Type each example yourself (do not copy and paste). Save, then refresh the browser.
3. Change something, break it on purpose, and see what happens. That is how the tags stick.

You need a browser (Chrome, Firefox, Edge, or Safari) and a text editor such as VS Code.

0. Your first page

Every HTML page starts with the same skeleton. `<!doctype html>` tells the browser this is HTML5, `lang` helps screen readers pick the right pronunciation, `charset` makes sure special characters display properly, and `viewport` makes the page behave on phones. Only what is inside `<body>` is visible.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
  </body>
</html>
```

Try it: change the `<title>` and watch the browser tab change. Change the `<h1>` text and refresh.

---

1. Text and structure

1.1 Headings and paragraphs

`h1` to `h6` create the outline of your page. Use one `h1` for the main topic, `h2` for sections, `h3` for sub-sections. Never pick a heading just because it looks bigger; that is a job for CSS. `p` holds a paragraph of text.

```
<h1>Lagos Food Guide</h1>
<p>A short guide to eating well in the city.</p>

<h2>Street food</h2>
<p>Suya, roasted plantain, and akara are everywhere.</p>
```

```
<h3>Suya</h3>
<p>Grilled spiced beef, usually served with onions and pepper.</p>

<h2>Restaurants</h2>
<p>From casual buka spots to fine dining.</p>
```

1.2 Emphasis and small text tags

These tags mark up words inside a paragraph. Use `strong` for important text, `em` for stressed text, `mark` for a highlight, `small` for fine print, `del` and `ins` for removed and added text, and `sub` and `sup` for lowered and raised characters.

```
<p><strong>Warning:</strong> the form closes on Friday.</p>
<p>You <em>must</em> bring your ID card.</p>
<p>Today's special: <mark>jollof rice</mark> with chicken.</p>
<p>Price: <del>₦5,000</del> <ins>₦4,000</ins></p>
<p>Water is H<sub>2</sub>O and the area is 5m<sup>2</sup>.</p>
<p><small>&copy; 2026 My Company. All rights reserved.</small></p>
```

Note: `b`, `i`, and `u` only change the look (bold, italic, underline) with no extra meaning. Prefer `strong` and `em`.

1.3 Line breaks and dividers

`br` forces a new line, and is correct for addresses and poems. `hr` draws a divider between topics. Do not use `br` just to add space; use CSS for spacing.

```
<p>
  Sonex Studio<br>
  12 Allen Avenue<br>
  Ikeja, Lagos
</p>

<hr>

<p>Next topic starts here.</p>
```

1.4 Links

The `a` tag creates a link. `href` is the destination. Use `target="_blank"` to open a new tab, and add `rel="noopener"` when you do. Link text should describe the destination, so avoid "click here".

```
<!-- Another website -->
<a href="https://developer.mozilla.org" target="_blank" rel="noopener">Read the
MDN web docs</a>

<!-- Another page in your folder -->
```

```
<a href="about.html">About us</a>

<!-- Jump to a section on this page -->
<a href="#contact">Go to contact section</a>
<h2 id="contact">Contact</h2>

<!-- Email and phone -->
<a href="mailto:hello@example.com">Email us</a>
<a href="tel:+2348000000000">Call us</a>
```

1.5 Lists

Use `ul` for bullet lists (order does not matter), `ol` for numbered lists (order matters), and `dl` for term and description pairs. Items go inside `li` (or `dt`/`dd` for description lists).

```
<h2>Shopping list</h2>
<ul>
  <li>Rice</li>
  <li>Tomatoes</li>
  <li>Fish</li>
</ul>

<h2>How to make tea</h2>
<ol>
  <li>Boil water.</li>
  <li>Add a tea bag.</li>
  <li>Wait three minutes.</li>
</ol>

<h2>Glossary</h2>
<dl>
  <dt>HTML</dt>
  <dd>The language that gives a web page its structure.</dd>
  <dt>CSS</dt>
  <dd>The language that styles a web page.</dd>
</dl>
```

Lists can be nested by placing a new `ul` or `ol` inside an `li`:

```
<ul>
  <li>Fruits
    <ul>
      <li>Mango</li>
      <li>Orange</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```

1.6 Tables

Tables are for data that belongs in rows and columns, never for page layout. `caption` titles the table, `thead` holds the header row, `tbody` holds the data, `tr` is a row, `th` is a header cell, and `td` is a data cell. `scope="col"` tells screen readers the header applies to a column.

```
<table border="1">
  <caption>Class timetable</caption>
  <thead>
    <tr>
      <th scope="col">Day</th>
      <th scope="col">Subject</th>
      <th scope="col">Time</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Monday</td>
      <td>Mathematics</td>
      <td>8:00 AM</td>
    </tr>
    <tr>
      <td>Tuesday</td>
      <td>English</td>
      <td>9:30 AM</td>
    </tr>
  </tbody>
</table>
```

To merge cells, use `colspan` (across columns) or `rowspan` (across rows):

```
<tr>
  <td colspan="3">Break time</td>
</tr>
```

1.7 Quotes and code

`blockquote` is a long quotation, `q` is a short inline one, `cite` names the work, `code` marks a piece of code inside text, and `pre` keeps spacing and line breaks exactly as typed.

```
<blockquote cite="https://example.com/speech">
  <p>The best way to learn is to build something.</p>
</blockquote>
<p>Source: <cite>A Developer's Notebook</cite></p>

<p>As the saying goes, <q>practice makes perfect</q>.</p>

<p>Use the <code>&lt;p&gt;</code> tag for paragraphs.</p>
```

```
<pre><code>function add(a, b) {
  return a + b;
}</code></pre>
```

To show a `<` or `>` on the page, type `<` and `>`, otherwise the browser reads it as a tag.

1.8 Comments

Comments are notes for you. The browser ignores them.

```
<!-- TODO: replace this paragraph with real content -->
<p>Placeholder text.</p>
```

1.9 Semantic layout

Semantic tags describe the purpose of each region. They look the same as `div` by default, but they help screen readers, search engines, and other developers. Here is a complete page layout using all of them:

```
<body>
  <header>
    <h1>My Tech Blog</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="posts.html">Posts</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>Why HTML Matters</h2>
      <p>Every website starts with good structure.</p>

      <section>
        <h3>Accessibility</h3>
        <p>Semantic tags help everyone use your page.</p>
      </section>

      <section>
        <h3>SEO</h3>
        <p>Search engines read your structure too.</p>
      </section>
    </article>

    <aside>
      <h2>Related posts</h2>
      <ul>
        <li><a href="#">Learning CSS</a></li>
```

```
        <li><a href="#">Choosing a code editor</a></li>
      </ul>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 My Tech Blog</p>
  </footer>
</body>
```

Quick guide: `header` is the top, `nav` holds main links, `main` is the unique content (only one per page), `article` is a self-contained piece, `section` groups related content under a heading, `aside` is side content, and `footer` is the bottom.

1.10 Generic containers: div and span

When no semantic tag fits, use `div` (a block) or `span` (inline). Their main use is to be styled later with CSS using `class`.

```
<div class="card">
  <h3>Product name</h3>
  <p>This product has a <span class="highlight">special offer</span> today.</p>
</div>
```

Practice 1: Build a profile page

Build a page about yourself that uses: one `h1`, two `h2` sections, a paragraph with a `strong` word, an `ol` of your goals, a table of your weekly schedule, and a `footer` with a link to your email. Wrap everything in `header`, `main`, and `footer`.

---

2. Images and media

2.1 Images

`img` has no closing tag. `src` is the file path or URL, `alt` describes the image for screen readers and shows if the image fails to load, and `width` and `height` let the browser reserve space so the page does not jump while loading.

```
<img src="images/campus.jpg"
     alt="Students walking beside the campus library"
     width="800" height="450">
```

Paths matter. Given this folder structure:

```
html-practice/
  index.html
  images/
    campus.jpg
```

`images/campus.jpg` works from `index.html`. If `index.html` were inside a sub-folder, you would need `../images/campus.jpg` to step up one level.

For a purely decorative image, leave `alt` empty so screen readers skip it:

```
<img src="images/divider.png" alt="">
```

An image can also be a link:

```
<a href="products.html">
  <img src="images/shop.jpg" alt="Browse our products" width="300" height="200">
</a>
```

2.4 Audio

`audio` plays sound. Add `controls` so the user can play, pause, and change volume. Multiple `source` tags give the browser format choices. Text inside the tag shows only in very old browsers.

```
<audio controls  src="media/lesson.mp3" type="audio/mpeg"> </audio>
```

It is good practice to provide a transcript nearby:

```
<details>
  <summary>Read the transcript</summary>
  <p>Welcome to lesson one. Today we will learn about HTML...</p>
</details>
```

2.5 Video

`video` works like `audio`, with extras: `poster` shows a preview image before playing, and `width` sets the size. The `track` tag adds captions.

```
<video controls width="640" poster="images/preview.jpg">
  <source src="media/lesson.mp4" type="video/mp4">
  <source src="media/lesson.webm" type="video/webm">
```

```
  <track kind="captions" src="media/lesson-en.vtt" srclang="en" label="English">
  Your browser does not support the video element.
</video>
```

Other useful attributes: `loop` (repeat), `muted` (start silent), and `preload="none"` (do not download until played). Avoid `autoplay` with sound, because it annoys users and browsers often block it.

2.6 Iframes

`iframe` shows another web page inside yours, such as a map or a video from another site. Always add a `title` so screen readers can describe it. Use `loading="lazy"` so it loads only when scrolled into view.

```
<iframe src="https://www.openstreetmap.org/export/embed.html?
bbox=3.30,6.40,3.45,6.55"
        title="Map of Lagos"
        width="600" height="400"
        loading="lazy"></iframe>
```

Note: some sites refuse to be embedded. If you see a blank frame, that site has blocked it.

Practice 2: Build a gallery page

Create a page with a `figure` containing an image and caption, a `picture` that swaps images by screen width, a short `video` with a poster, and an `audio` clip with a transcript inside `details`. Check that every image has meaningful `alt` text.

---

3. Forms

3.1 The form container

`form` wraps all the controls. `action` is the URL that receives the data and `method` is how it is sent: `get` puts data in the URL (good for search), `post` sends it in the request body (good for logins and sign-ups).

```
<form action="/submit" method="post">
  <!-- controls go here -->
</form>
```

When testing without a server, you can leave `action="#"`. The page will simply reload.

3.2 Labels and text inputs

Every input needs a `label`. Link them with `for` (on the label) matching `id` (on the input). Clicking the label then focuses the input, and screen readers announce it. `name` is the key that is sent with the data, so an input without a `name` is not submitted.

```
<form action="#" method="post">
  <label for="fullname">Full name</label>
  <input type="text" id="fullname" name="fullname" placeholder="Ada Obi" required>

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="password">Password</label>
  <input type="password" id="password" name="password" minlength="8" required>

  <button type="submit">Create account</button>
</form>
```

Try submitting with the email empty, then with `abc` typed in. The browser validates for you because of `required` and `type="email"`.

3.3 More input types

The `type` attribute changes the control and the keyboard shown on phones.

```
<label for="phone">Phone</label>
<input type="tel" id="phone" name="phone" placeholder="0801 234 5678">

<label for="site">Website</label>
<input type="url" id="site" name="site" placeholder="https://">

<label for="q">Search</label>
<input type="search" id="q" name="q">

<label for="qty">Quantity (1 to 10)</label>
<input type="number" id="qty" name="qty" min="1" max="10" step="1" value="1">

<label for="volume">Volume</label>
<input type="range" id="volume" name="volume" min="0" max="100" value="50">

<label for="dob">Date of birth</label>
<input type="date" id="dob" name="dob">

<label for="appt">Appointment</label>
<input type="datetime-local" id="appt" name="appt">

<label for="alarm">Alarm time</label>
<input type="time" id="alarm" name="alarm">

<label for="color">Favourite colour</label>
<input type="color" id="color" name="color" value="#4f46e5">
```

3.4 Validation attributes

HTML can check input before it is sent: `required` (cannot be empty), `minlength` and `maxlength` (text length), `min` and `max` (number or date range), and `pattern` (a regular expression the value must match). Add a `title` to explain the pattern.

```
<label for="username">Username (letters and numbers, 4 to 12 characters)</label>
<input type="text" id="username" name="username"
       pattern="[A-Za-z0-9]{4,12}"
       title="4 to 12 letters or numbers, no spaces"
       required>

<label for="age">Age (18 to 65)</label>
<input type="number" id="age" name="age" min="18" max="65" required>
```

Important: browser validation is only a convenience. Always validate again on the server, because users can bypass the browser.

3.5 Checkboxes and radio buttons

Use checkboxes when many options can be chosen, and radio buttons when only one can. Radio buttons with the same `name` form a group. Each `value` is what gets sent. Wrap the group in `fieldset` with a `legend` so screen readers announce the question.

```
<fieldset>
  <legend>Which languages do you know?</legend>

  <input type="checkbox" id="html" name="skills" value="html" checked>
  <label for="html">HTML</label>

  <input type="checkbox" id="css" name="skills" value="css">
  <label for="css">CSS</label>

  <input type="checkbox" id="php" name="skills" value="php">
  <label for="php">PHP</label>
</fieldset>

<fieldset>
  <legend>Preferred contact method</legend>

  <input type="radio" id="c-email" name="contact" value="email" required>
  <label for="c-email">Email</label>

  <input type="radio" id="c-phone" name="contact" value="phone">
  <label for="c-phone">Phone</label>
</fieldset>
```

3.6 Drop-down lists

`select` shows a drop-down, `option` is each choice, and `optgroup` groups choices. Add `multiple` to allow more than one selection.

```
<label for="country">Country</label>
<select id="country" name="country" required>
  <option value="">Select a country</option>
    <option value="ng">Nigeria</option>
    <option value="gh">Ghana</option>
    <option value="ke">Kenya</option>
    <option value="tz">Tanzania</option>
</select>
```

`datalist` gives suggestions while still letting the user type anything:

```
<label for="browser">Favourite browser</label>
<input list="browsers" id="browser" name="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

3.7 Multi-line text

`textarea` is for longer text. `rows` sets the visible height and `maxlength` limits the length. Unlike `input`, it has a closing tag, and default text goes between the tags.

```
<label for="message">Your message</label>
<textarea id="message" name="message" rows="5" cols="40" maxlength="500"
          placeholder="Write your message here..."></textarea>
```

3.8 File uploads

For file uploads, the form must use `method="post"` and `enctype="multipart/form-data"`. Without `enctype`, only the file name is sent, not the file. `accept` limits the file types and `multiple` allows several files.

```
<form action="/upload" method="post" enctype="multipart/form-data">
  <label for="avatar">Profile picture</label>
  <input type="file" id="avatar" name="avatar" accept="image/*">

  <label for="docs">Supporting documents (PDF only)</label>
  <input type="file" id="docs" name="docs[]" accept=".pdf" multiple>
```

```
  <button type="submit">Upload</button>
</form>
```

The `docs[]` name lets PHP receive several files as an array. As with validation, `accept` is only a hint, so check the file type again on the server.

3.9 Hidden fields and buttons

A hidden input sends a value the user does not see, such as an ID. There are three button types: `submit` sends the form, `reset` clears it, and `button` does nothing on its own (it needs JavaScript, which is outside this guide).

```
<input type="hidden" name="form_id" value="signup-2026">

<button type="submit">Send</button>
<button type="reset">Clear form</button>
```

You can disable a button with `disabled`, and give a form control a starting value with `value`.

3.11 Complete example: a registration form

This form combines everything from this section. Type it out, then submit it with fields empty and filled to see the validation at work.

```
<form action="/register" method="post" enctype="multipart/form-data">
  <fieldset>
    <legend>Create your account</legend>

    <p>
      <label for="r-name">Full name</label><br>
      <input type="text" id="r-name" name="fullname" required>
    </p>

    <p>
      <label for="r-email">Email</label><br>
      <input type="email" id="r-email" name="email" required>
    </p>

    <p>
      <label for="r-pass">Password (minimum 8 characters)</label><br>
      <input type="password" id="r-pass" name="password" minlength="8" required>
    </p>

    <p>
      <label for="r-dob">Date of birth</label><br>
      <input type="date" id="r-dob" name="dob" required>
    </p>

    <p>
```

```
      <label for="r-level">Level</label><br>
      <select id="r-level" name="level" required>
        <option value="">Choose level</option>
        <option value="beginner">Beginner</option>
        <option value="intermediate">Intermediate</option>
        <option value="advanced">Advanced</option>
      </select>
    </p>

    <fieldset>
      <legend>Gender</legend>
      <input type="radio" id="g-f" name="gender" value="female" required>
      <label for="g-f">Female</label>
      <input type="radio" id="g-m" name="gender" value="male">
      <label for="g-m">Male</label>
    </fieldset>

    <p>
      <label for="r-photo">Profile photo</label><br>
      <input type="file" id="r-photo" name="photo" accept="image/*">
    </p>

    <p>
      <label for="r-bio">Short bio</label><br>
      <textarea id="r-bio" name="bio" rows="4" maxlength="300"></textarea>
    </p>

    <p>
      <input type="checkbox" id="r-terms" name="terms" value="yes" required>
      <label for="r-terms">I agree to the terms and conditions</label>
    </p>

    <button type="submit">Register</button>
    <button type="reset">Clear</button>
  </fieldset>
</form>
```

Practice 3: Build a contact form

Build a contact form with name, email, a subject `select`, a message `textarea`, a radio group for preferred reply method, and a required consent checkbox. Then confirm that every input has a matching `label`, that every field has a `name`, and that you can complete the whole form using only the Tab key.

---

4. Final project: a one-page site

Combine all three sections in one `index.html`: a `header` with a `nav` of anchor links, a `main` with an `article` (headings, paragraphs, a list, and a table), a `figure` with an image, a `video`, and a contact form, and a `footer`. If your page can be read top to bottom with correct headings, every image has `alt` text, and every input has a label, you have the foundations of HTML.

Quick reference

Need	Tag
Page title and sections	`h1` to `h6`, `section`, `article`
Text	`p`, `strong`, `em`, `mark`, `code`, `pre`
Links	`a` with `href`
Lists	`ul`, `ol`, `li`, `dl`
Data in rows and columns	`table`, `thead`, `tbody`, `tr`, `th`, `td`
Images	`img`, `figure`, `picture`
Media	`img`,`audio`, `video`, `iframe`
Forms	`form`, `label`, `input`, `select`, `textarea`, `button`

What next?

Learn CSS to style your pages, then JavaScript when you want behaviour and interactivity.